UNIVERSITY INSTITUTE OF COMPUTING

Discover. Learn. Empower.

NAAC GRADE A+ Accredited University

CHANDIGARH UNIVERSITY

University Institute of Computing

MINI PROJECT REPORT

ON

UNIVERSITY CAMPUS EVENT & STUDENT ACTIVITY MANAGEMENT PORTAL

(UniVentPortal)

Advanced Internet Programming (AIP)

(26CAH-614)

Submitted By:

Arpit Saxena

26MCD10006

MCA (Data Science)

26MCD-1 (A)

Submitted To:

Ms. Tanisha

E20679

Master of Computer Applications (MCA)

University Institute of Computing

Chandigarh University,

Gharuan, Mohali, Punjab, India-140413

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Certificate	2
2	Declaration	2
3	Acknowledgement	3
4	Abstract	3
5	Introduction	4
6	Problem Statement	4
7	Objectives	4
8	Scope of the Project	4
9	Existing and Proposed System	5
10	Technologies Used	5
11	System Requirements	5–6
12	System Architecture	6–7
13	Database Design	7
14	Project Modules	7
15	Implementation	7
16	Important Code Snippets	8–11
17	Testing and Results	12
18	Output Screenshots	12–14
19	Conclusion and Future Scope	14
20	References	14–15

UNIVERSITY INSTITUTE OF COMPUTING

Discover. Learn. Empower.

NAAC GRADE A+ Accredited University

CERTIFICATE

This is to certify that **Arpit Saxena**, student of **Master of Computer Applications (MCA), Data Science, Semester 1**, has undertaken the mini project entitled **"University Campus Event & Student Activity Management Portal (UniVentPortal)"** as part of the **Advanced Internet Programming** course during the academic session 2026–27.

The project demonstrates the application of Java web technologies, relational database connectivity, session-based authentication and event registration workflows in an integrated web application.

Student Signature: _____

Faculty Signature: _____

Date: _____

DECLARATION

I, **Arpit Saxena**, hereby declare that the mini-project entitled **"University Campus Event & Student Activity Management Portal (UniVentPortal)"** is my original academic work, carried out as part of the **Master of Computer Applications (MCA) – Data Science, 1st Semester** programme at Chandigarh University, Mohali.

The project has been developed as part of the **Advanced Internet Programming (AIP)** course during the academic session **2026–2027**. The work presented in this report reflects my understanding and practical application of Java programming, JSP, Servlets, JDBC, MySQL database management, web application development, and session-based authentication.

I hereby declare that this project report has been prepared for academic purposes and that the work presented is submitted in partial fulfillment of the academic requirements of the MCA (Data Science) programme.

Student Signature: _____

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Ms. Tanisha (E20679)** for her valuable guidance, support, and encouragement throughout the development of my mini-project, **"University Campus Event & Student Activity Management Portal (UniVentPortal)."** Her guidance and suggestions helped me understand the practical application of Java programming, JSP, Servlets, JDBC, MySQL database management, and web application development.

I am also thankful to **Chandigarh University** and the **University Institute of Computing (UIC)** for providing the necessary academic environment, facilities, and resources to complete this project successfully as part of the MCA (Data Science) programme.

I would also like to thank my teachers and everyone who supported me directly or indirectly during the development and testing of this project. This project has been a valuable learning experience, allowing me to enhance my programming skills and apply theoretical knowledge to a practical campus event management system.

ABSTRACT

The **University Campus Event & Student Activity Management Portal (UniVentPortal)** is a Java-based web application designed to simplify campus event management by providing a centralized platform for discovering, publishing, and registering for university events. The system enables students to explore upcoming activities, view event details, manage their registrations, and monitor their registered events efficiently.

The application is developed using **Java, JSP, Servlets, JDBC, HTML, CSS, JavaScript, and MySQL**. It integrates database connectivity through JDBC and uses MySQL for storing user accounts, event details, and registration records. The application provides separate interfaces for students and administrators, allowing students to register for events and administrators to publish events, manage active events, monitor registration capacity, and view registered-student details.

The system also supports essential features such as student sign-up, session-based authentication, event category filtering, registration cancellation, and database-backed record management. These features help organize campus activities, reduce manual administrative work, and improve access to event information.

The main objective of this project is to demonstrate the practical application of Java web technologies, database management, server-side programming, and web application development in solving real-world campus event management problems. The proposed system offers a structured and user-friendly solution for managing university events and student participation.

1. INTRODUCTION

University campuses regularly conduct technical events, workshops, cultural festivals, sports activities and student development programmes. When event information is distributed through separate notices, messages or spreadsheets, students may find it difficult to locate relevant activities and administrators may need to manage registrations manually.

UniVentPortal provides a centralized online portal where students can explore campus activities and administrators can manage event information. The application is built as a Java web project using JSP, Jakarta Servlets, JDBC and MySQL, following an MVC Model 2 approach. The browser communicates with the servlet controller, which coordinates the application logic and data operations before returning a JSP view.

2. PROBLEM STATEMENT

The main problem addressed by the project is the lack of a single, organized interface for event discovery and registration. Manual processes can make event details inconsistent, make it harder to track capacity, and require additional effort to maintain participant lists.

The proposed system stores users, events and registrations in a relational database. It provides event filtering, controlled sign-in, student registration and cancellation, and administrator tools for publishing events and reviewing enrolment. Database constraints help prevent duplicate registrations and maintain relationships between users, events and registration records.

Need for the System

- Provide one place to browse campus events and their details.
- Reduce manual effort in maintaining event listings and participant rosters.
- Track registration counts against event capacity.
- Separate student functions from administrator functions.
- Maintain persistent records using a relational database.

3. OBJECTIVES

- Develop a web-based portal for discovering and managing campus events.
- Provide student account creation and session-based sign-in.
- Allow students to register for events and review their registered activities.
- Support registration cancellation and capacity updates.
- Allow administrators to publish and remove events.
- Display event enrolment information and registered-student rosters.
- Use JDBC and MySQL to store and retrieve application data.
- Demonstrate servlet routing, JSP views, form handling and relational constraints.

4. SCOPE OF THE PROJECT

The current project focuses on campus event discovery and registration. The public page lists events and offers category filters. Student functions include sign-up, login, viewing registered activities and cancelling a booking. Administrator functions include publishing events, deleting active events and inspecting the attendee roster.

The project is an academic mini project rather than a complete university-wide production deployment. Its design can be extended with email notifications, QR-code scanning, attendance tracking, analytics, event approval workflows and additional administrative reporting.

5. EXISTING AND PROPOSED SYSTEM

Existing / Manual Approach	Proposed UniVentPortal
Event information may be scattered across notices and messages.	Events are presented in one searchable/filterable catalogue.
Participant lists may require manual updates.	Registrations are stored in the database and shown in dashboards.
Capacity can be difficult to track consistently.	Each event stores a capacity and registration count.
Student and admin tasks may use separate processes.	Role-specific student and administrator views are provided.

6. TECHNOLOGIES USED

Technology	Purpose in the Project
Java (JDK 17+)	Core server-side language and application logic.
Jakarta Servlets	Handle requests, authentication, routing and event operations.
JSP	Render the public catalogue, authentication forms and dashboards.
HTML5 / CSS3	Structure and style the web interface.
JavaScript	Client-side form validation and interface helpers.
JDBC	Connect Java code to the MySQL database.
MySQL / XAMPP	Persist user profiles, events and registrations.
Apache Tomcat 10.1+	Servlet/JSP runtime web container.
Apache NetBeans + Maven	Development, project configuration and build/deployment.
GitHub	Version control and public project repository.

7. SYSTEM REQUIREMENTS

Hardware	Software
Processor: Intel Core i3 or equivalent	Operating System: Windows or compatible OS
RAM: 4 GB minimum; 8 GB recommended	JDK 17 or later
At least 1 GB free project/database space	Apache NetBeans IDE
Keyboard, mouse and display	Apache Tomcat 10.1+
Internet connection for repository access	XAMPP / MySQL and a modern browser

Local development URL: http://localhost:8080/index.jsp (as shown during local testing). The README also describes deployment under the CampusEventPortal context path, depending on the configured application name.

8. SYSTEM ARCHITECTURE

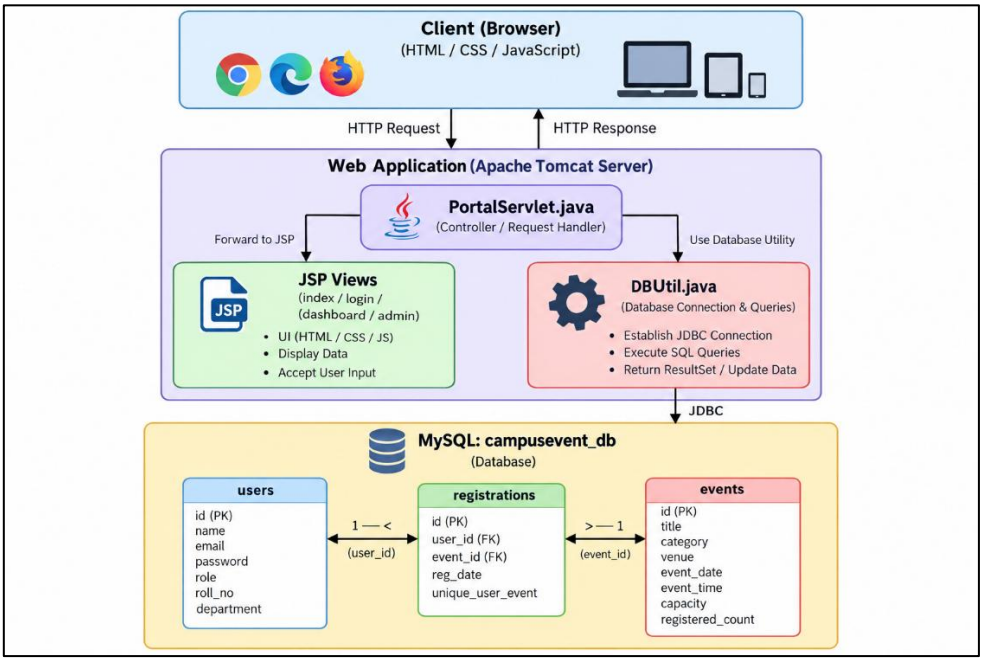
The application follows the MVC Model 2 pattern. JSP pages act as views, PortalServlet.java acts as the main controller, and DBUtil.java provides the database connection utility. The MySQL database stores users, events and registrations.

Layer / Component	Responsibility
Client Browser	Displays pages and submits HTTP form requests.
JSP Views	index.jsp: public event catalogue; login.jsp: sign-in/sign-up; dashboard.jsp: student activities; admin.jsp: administration.
PortalServlet.java	Processes authentication, session checks, event creation/deletion, registration and cancellation workflows.
DBUtil.java / JDBC	Creates database connections and supports parameterized SQL operations.
MySQL: campusevent_db	Stores user accounts, event details and event registrations.

Request and Data Flow

1. A student or administrator opens a JSP page in the browser.
2. The browser submits a form or sends a request to the servlet controller.
3. The controller validates the request and checks the current session/role when required.
4. For data operations, JDBC executes parameterized SQL against MySQL.
5. The controller prepares a response or forwards the request to a JSP view.
6. The browser displays the updated event list, dashboard or administration table.

Architecture Diagram



The separation of presentation, request handling and data access makes the application easier to maintain than placing all logic directly inside individual pages.

9. DATABASE DESIGN

The database is named campusevent_db and contains three related tables: users, events and registrations. The registrations table acts as a junction table between users and events. Foreign keys preserve referential integrity, while a unique constraint on the user/event pair prevents the same user from registering for the same event more than once.

Table	Important Fields	Purpose
users	id (PK), name, email (UNIQUE), password, role, roll_no, department	Stores administrator and student account details.
events	id (PK), title, category, venue, event_date, event_time, capacity, registered_count	Stores event descriptions, schedule and capacity.
registrations	id (PK), user_id (FK), event_id (FK), reg_date; UNIQUE(user_id,event_id)	Links users to events they have registered for.

Relationship Summary

One user can have multiple registrations, and one event can have multiple registrations. Each registration references exactly one user and one event. The schema defines cascading deletion for related registration rows when a user or event is deleted.

Database Configuration

```
CREATE DATABASE IF NOT EXISTS campusevent_db;
USE campusevent_db;

-- Core entities
CREATE TABLE IF NOT EXISTS users (...);
CREATE TABLE IF NOT EXISTS events (...);
CREATE TABLE IF NOT EXISTS registrations (...);

-- Prevent duplicate bookings
UNIQUE (user_id, event_id)
```

The complete schema and sample records are maintained in database/schema.sql in the GitHub repository. The SQL above is a shortened illustrative excerpt, not a replacement for the full schema file.

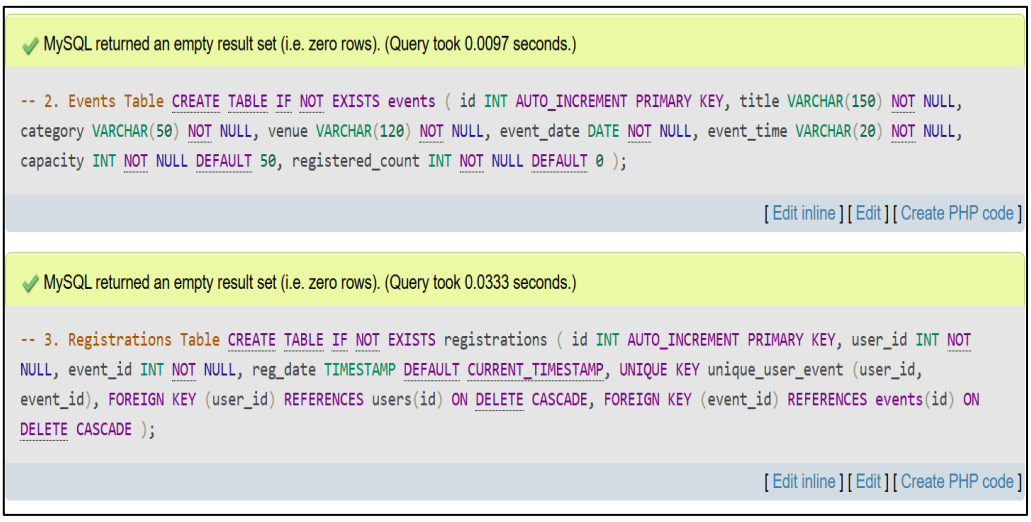


Figure 1. MySQL schema / sample-data setup in phpMyAdmin.

10. PROJECT MODULES

Public Event Explorer

The public home page presents the event catalogue with category filters for All, Technical, Cultural, Workshop and Sports. Event cards show the title, category, date/time, venue, capacity or seats remaining, and a sign-in prompt for registration.

Authentication and Account Creation

The authentication page combines Sign-In and Student Sign-Up. The form collects relevant account information and sends it to the server for validation. The README documents demo administrator and student accounts for local testing.

Student Dashboard

After authentication, a student can view a personalized dashboard containing their profile details and registered activities. The dashboard includes event information and a cancellation action for a registration.

Administrator Dashboard

The administrator panel provides a form for publishing new events, a table of active events with enrolment/capacity values, event deletion controls and a registered-students roster.

Registration and Capacity Handling

The registration workflow associates a user with an event, prevents duplicate user/event pairs and updates the registration count. Cancellation releases the registration and updates the available capacity. The data model and server-side logic are intended to keep registration records consistent.

Important Project Files

File / Folder	Role
src/java/com/campusevent/PortalServlet.java	Main controller for request routing and business workflows.
src/java/com/campusevent/DBUtil.java	Database connection helper.
web/index.jsp	Public event catalogue.
web/login.jsp	Sign-in and sign-up interface.
web/dashboard.jsp	Student activity dashboard and pass view.
web/admin.jsp	Administrator event and roster interface.
database/schema.sql	Database schema and seeded demo records.

11. CODE SNIPPETS

Only selected excerpts are included below to explain the main concepts. The complete source code remains in the public GitHub repository.

JDBC Connection

```
Class.forName("com.mysql.cj.jdbc.Driver");
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/campusevent_db",
    "root",
    ""
);
```

DBUtil.java centralizes connection setup so that controller code does not need to repeat the database configuration in every operation.

Parameterized Query

```
String sql = "SELECT * FROM users WHERE email = ?";
PreparedStatement ps = con.prepareStatement(sql);
ps.setString(1, email);
ResultSet rs = ps.executeQuery();
```

PreparedStatement binds user-provided values separately from SQL text. This reduces the risk of SQL injection compared with concatenating raw form input into a query.

Session-Based Access Check

```
HttpSession session = request.getSession(false);
if (session == null || session.getAttribute("user") == null) {
    response.sendRedirect("login.jsp");
    return;
}
```

A session check prevents protected pages from being used without an authenticated session. The exact session attribute names should match the implementation in PortalServlet.java.

Registration Uniqueness

```
UNIQUE (user_id, event_id)
FOREIGN KEY (user_id) REFERENCES users(id)
  ON DELETE CASCADE,
FOREIGN KEY (event_id) REFERENCES events(id)
  ON DELETE CASCADE
```

The unique pair prevents duplicate registrations at the database level. Foreign keys link each registration to a valid user and event.

Event Insertion (Illustrative)

```
String sql = "INSERT INTO events (...) VALUES (?, ?, ?, ?, ?, ?)";
```

12. TESTING AND RESULTS

The project was reviewed through the locally running interface and database setup. The following functional test cases describe the expected behaviour of the main screens and workflows. Results should be rechecked after any future source-code changes or when deploying to another environment.

Test Case	Action / Input	Expected Outcome
Public catalogue	Open the home page.	Event cards and category filters are displayed.
Category filter	Choose Technical / Cultural / Workshop / Sports.	Catalogue narrows to the selected category.
Student sign-up	Submit valid student details.	Account is validated and created if the details are acceptable.
Valid sign-in	Enter a seeded student account.	Student dashboard opens with profile/registration information.
Admin sign-in	Enter the seeded administrator account.	Administrator panel becomes available.
Publish event	Submit title, category, venue, date, time and capacity.	New event appears in the active-events list.
Register for event	Select an available event while signed in.	Registration is recorded and capacity information updates.
Duplicate registration	Try registering for the same event again.	Duplicate user/event registration is rejected.
Cancel registration	Cancel an existing student registration.	Registration is removed and a seat is released.
Delete event	Use the administrator delete action.	Event is removed and dependent registration rows follow database constraints.

Database Testing

- Verify that users, events and registrations tables exist in campusevent_db.
- Check that email addresses are unique in the users table.
- Check that registrations reference valid user and event IDs.
- Confirm the unique (user_id, event_id) constraint prevents duplicate bookings.
- Verify event counts and student rosters after registration or cancellation.

Result Summary

The captured application screens show the public event catalogue, authentication page, administrator event-management interface and student registered-activities dashboard. These screens demonstrate the intended user-facing flow and the project's core modules. Database screenshots also show the schema/sample-data setup in phpMyAdmin.

13. OUTPUT

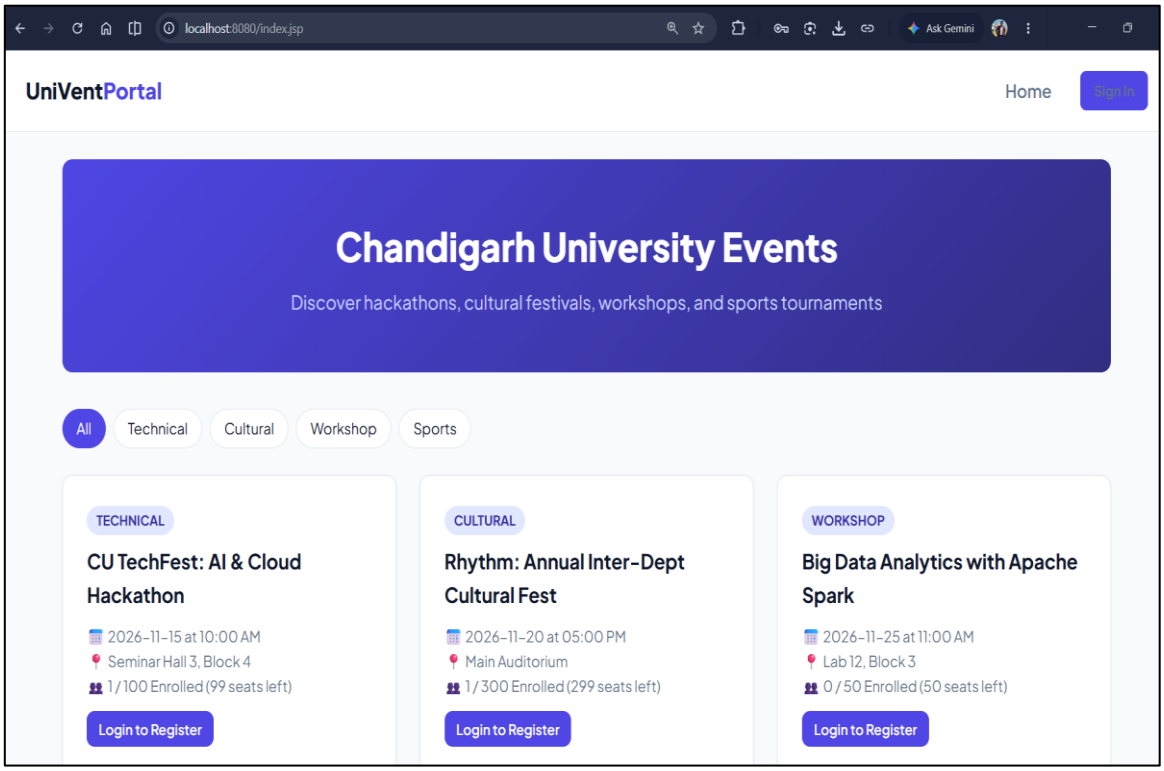


Figure 2. Public home page with event cards and category filters.

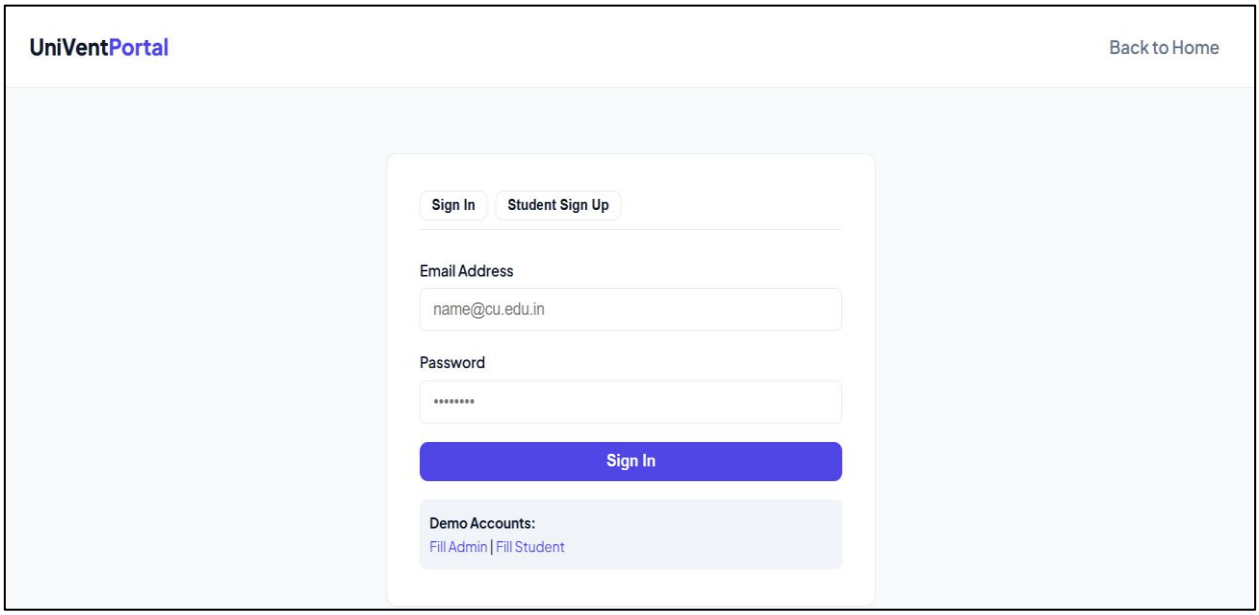


Figure 3. Sign-in screen with demo-account helper links.

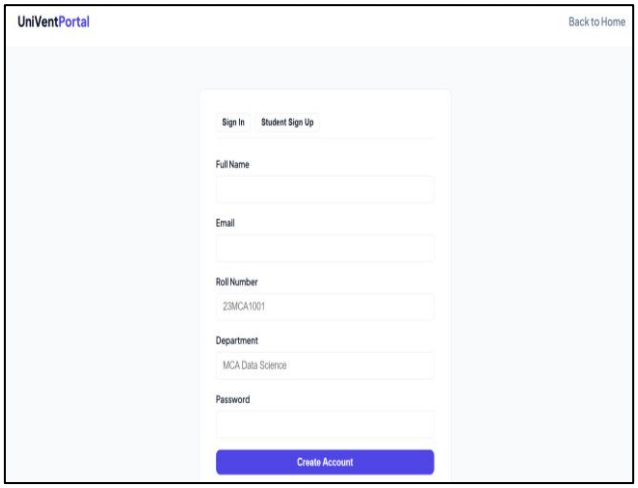


Figure 4. Student sign-up form.

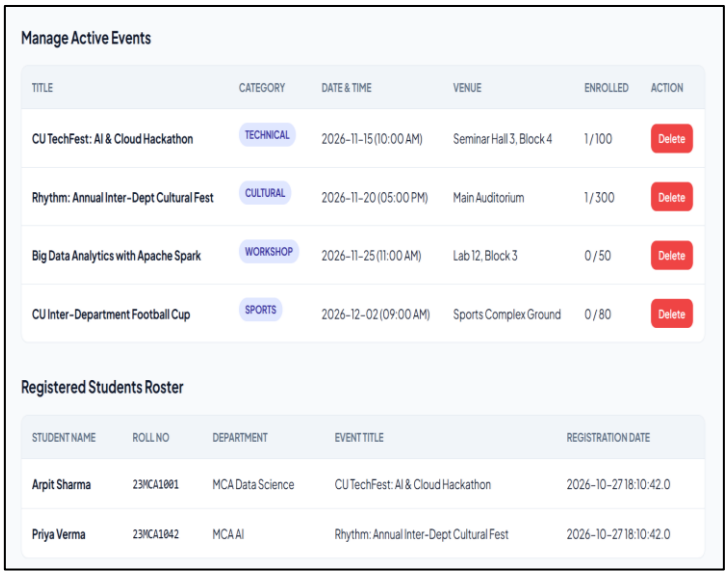


Figure 5. Administrator panel showing active events and registered-student roster.



Figure 6. Student dashboard showing registered activities and cancellation action.

14. CONCLUSION

UniVentPortal brings campus event discovery, student registration and administrator event management into one Java web application. The project combines JSP views, servlet-based request handling, JDBC connectivity and a MySQL relational database. It demonstrates a practical MVC Model 2 structure and illustrates how application logic and database constraints can support event registration workflows.

The public catalogue makes event information easier to browse, while the student dashboard provides a place to review registered activities. The administrator panel supports event publication, enrolment monitoring and attendee-list review. The project also provided practical experience with NetBeans, Maven, Tomcat, XAMPP/phpMyAdmin, Git and GitHub.

15. FUTURE SCOPE

- Add email notifications for registration confirmations, cancellations and event reminders.
- Introduce event approval and publishing workflows for clubs or faculty organizers.
- Add QR-code based digital passes and attendance verification.
- Provide event analytics such as category popularity and participation trends.
- Improve account security with password hashing, stronger session controls and security auditing.
- Add pagination, advanced search, exportable attendee reports and accessibility improvements.
- Deploy the application to a secured hosting environment with managed database backups.

16. REFERENCES

1. Project repository: https://github.com/arpitsaxenawork/campus-event-portal
2. Project README.md — setup instructions, project structure, technology stack and database schema.
3. Jakarta Servlet documentation: https://jakarta.ee/specifications/servlet/
4. JavaServer Pages / Jakarta Pages documentation: https://jakarta.ee/specifications/pages/
5. MySQL Reference Manual: https://dev.mysql.com/doc/
6. Apache Tomcat documentation: https://tomcat.apache.org/
7. Apache NetBeans documentation: https://netbeans.apache.org/